

Proyecto 1: Sistema de monitore para cadena de frío

Diana Méndez 241341

El proyecto consiste en un sistema de monitoreo de temperatura para una cadena de frío utilizando un ESP32. El sistema obtiene la temperatura mediante un sensor LM35 y dependiendo del valor medido, genera diferentes respuestas visuales y mecánicas. La temperatura se muestra mediante tres displays de siete segmentos, mientras que un LED RGB indica el rango en el que se encuentra la temperatura y un servomotor cambia de posición según dicho rango. La medición se realiza cuando se presiona un botón. Una vez obtenida, el ESP32 procesa la temperatura y envía tanto su valor como la posición del servomotor a Adafruit IO, permitiendo visualizar los datos en tiempo real desde internet.

Componentes Utilizados:

- ESP32
- Sensor de temperatura LM35
- Servomotor de 180°
- LED RGB
- Tres displays de siete segmentos de cátodo común
- Tres transistores 2N2222
- Botón pulsador
- Resistencias de 220Ω
- Resistencia de 10kΩ
- Fuente externa de 5V
- Protoboard y cables de conexión
- Plataforma Adafruit IO para monitore e internet

Circuitos Utilizados

- **Sensor de temperatura LM35**

El LM35 es un sensor analógico cuya salida de voltaje es proporcional a la temperatura. El sensor entrega aproximadamente 10 mV por cada grado Celsius. La salida del sensor se conecta al pin ADC GPIO36 del ESP32. El ESP32 utiliza un ADC de 12 bits, por lo que puede representar valores desde 0 hasta 4095. Para obtener el voltaje medio se utiliza: $V = 3300/4096$, y luego $T = V/10$. En el programa esto se realiza mediante:

```
int adcVal = analogRead(sensor);  
float millivolt = adcVal * (ADC_VREF_mV / ADC_RESOLUTION);  
float tempC = millivolt / 10.0;
```

- **Circuito del LED RGB**

Se utilizó un LED RGB para representar visualmente diferentes estados de temperatura. Cada canal del LED se controla de manera independiente:

- Rojo → GPIO32
- Verde → GPIO33
- Azul → GPIO25

El ESP32 utiliza PWM para controlar la intensidad de cada componente del LED mediante valores entre 0 y 255. Con estos valores se pueden hacer mezclas entre ellos para crear nuevos colores. Los colores utilizados fueron los siguientes:

Temperatura	LED RGB
Medor a 23°C	Azul
23°C a 25 °C	Verde
25 °C a 27 °C	Amarillo
27 °C o mayor	Rojo

```
void apagarLED() {
  escribirRGB(0, 0, 0);
}

void encenderRojo() {
  escribirRGB(255, 0, 0);
}

void encenderVerde() {
  escribirRGB(0, 255, 0);
}
```

```
void encenderAzul() {
  escribirRGB(0, 0, 255);
}

void encenderGelb() {
  escribirRGB(255, 170, 0);
}
```

- **Circuito y funcionamiento del servomotor**

El servomotor se conecta al GPIO13 del ESP32 y es controlado mediante una señal PWM de 50 Hz. La posición del servo depende de la temperatura:

Temperatura	Posición del servo
Medor a 23°C	0°
23°C a 25 °C	45°
25 °C a 27 °C	45°
27 °C o mayor	90°

La señal utilizada por el servo varía entre 600µs (0°) y 2400µs (180°). En el código se convierten primero los grados deseados al ancho de pulso correspondiente. Luego el ancho del pulso se transforma al valor de duty cycle necesario para el PWM.

```
uint32_t pulso = map(grados, 0, 180, 600, 2400);
uint32_t duty = (pulso * 65535UL) / 20000UL;
```

- **Displays de siete segmentos**

El sistema posee tres displays de siete segmentos para representar la temperatura con un decimal. Por ejemplo, 24.7°C se separa internamente como:

- Decenas = 2
- Unidades = 4
- Decimales = 7

Esto se realiza con:

```
void separarTemperatura(float temperatura) {
    int temp = (int)(temperatura * 10.0 + 0.5);

    if (temp < 0) {
        temp = 0;
    }

    if (temp > 999) {
        temp = 999;
    }

    decenas = temp / 100;
    unidades = (temp / 10) % 10;
    decimal = temp % 10;
}
```

Cada número posee una combinación determinada de segmentos. Por ejemplo, {1, 1, 0, 1, 1, 0, 1} indica cuáles segmentos deben encenderse para formar el número 2. Los segmentos utilizados son:

- A → GPIO22
- B → GPIO23
- C → GPIO4
- D → GPIO17
- E → GPIO18
- F → GPIO21
- G → GPIO19

- **Multiplexado de los displays**

Debido a que los tres displays comparten las mismas líneas correspondientes a los segmentos a-g, se utiliza multiplexado. El ESP32 enciende únicamente un display a la vez durante aproximadamente 1 ms. El funcionamiento es:

- Display 1 → decenas → 1 ms
- Display 2 → unidades → 1 ms
- Display 3 → decimal → 1 ms

El cambio ocurre tan rápidamente que el ojo humano percibe los tres displays encendidos simultáneamente. Para evitar que el multiplexado sea interrumpido por las demás funciones del programa, se creó una tarea independiente de FreeRTOS:

```
// Tarea independiente para displays
xTaskCreatePinnedToCore(
    tareaDisplays,
    "Displays",
    2048,
    NULL,
    2,
    NULL,
    1
);
```

La tarea ejecuta constantemente mostrarNumero(decenas), mostrarNumero(unidades) y mostrarNumero(decimal) seleccionando el display correspondiente entre cada operación. Esto permite que el programa pueda conectarse a internet, mover el servo y procesar temperaturas sin detener la actualización visual de los displays.

- **Botón e interrupción**

La lectura de temperatura se activa utilizando un botón conectado al GPIO34. En lugar de revisar constantemente el estado del botón dentro del loop(), se utiliza una interrupción:

```
// Interrupcion
attachInterrupt(
  digitalPinToInterrupt((uint8_t)boton),
  botonISR,
  CHANGE
);
```

Cuando se detecta la pulsación se ejecuta `botonISR()`.

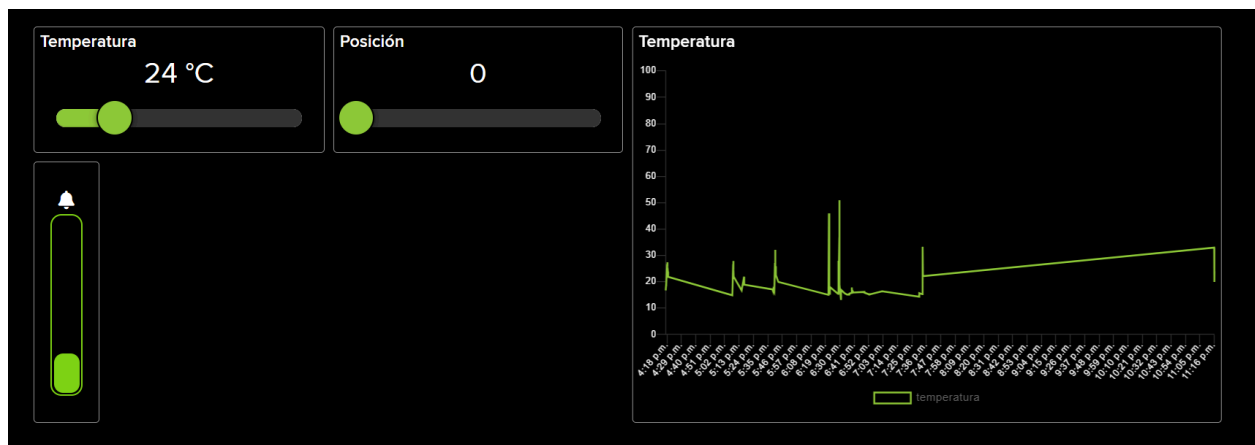
La interrupción no realiza directamente la medición, sino que activa una bandera `leerTemperatura = true;`. Posteriormente, el loop() detecta dicha bandera y realiza la lectura. Esto evita ejecutar operaciones largas dentro de la interrupción. También se implementó un antirrebote de 50 ms para evitar que una sola pulsación sea detectada varias veces debido al rebote mecánico del botón.

- **Funcionamiento general del programa**

Al encender el sistema, el ESP32 configura el sensor, botón, LED RGB, servomotor, displays y conexión con Adafruit IO. El funcionamiento puede resumirse de la siguiente manera

Encender sistema → Configurar componentes → Conectarse a Adafruit IO → Esperar botón → Leer ADC del LM35 → Convertir voltaje a °C → Separar temperatura para displays → Determinar rango de temperatura → Cambiar LED RGB → Mover servomotor → Mostrar temperatura → Enviar temperatura y servo a Adafruit IO → Esperar nueva medición.

- **Gráficos del Dashboard en Adafruit IO**



El eje horizontal representa el tiempo y el eje vertical representa la temperatura (°C). Los límites importantes del sistema son 23°C, 25°C y 27°C ya que estos valores representan los

puntos donde cambia el comportamiento del sistema. La gráfica permite observar las variaciones de temperatura y determinar cuándo el sistema cambia de estado. En Adafruit IO también se puede visualizar la posición del servo, permitiendo relacionar ambas variables.

- **Transmisión de datos a internet**

El ESP32 utiliza Adafruit IO para transmitir los datos obtenidos por el sistema. Se definieron dos feeds:

```
// Adafruit IO
AdafruitIO_Feed *canalTemperatura = io.feed("temperatura");
AdafruitIO_Feed *canalServoAdafruit = io.feed("servo");
```

Después de realizar una medición se envían los datos utilizando:

```
canalTemperatura->save(temperatura);
canalServoAdafruit->save(posicionServo);
```

Esto permite observar remotamente tanto la temperatura como la posición actual del servomotor. La función `io.run()` se ejecuta continuamente dentro del `loop()` para mantener activa la comunicación con Adafruit IO.

- **Código**

El programa fue dividido en diferentes funciones para facilitar su organización y comprensión. Las funciones principales son:

- `leerTemp()`; Lee el ADC y convierte la señal del LM35 a grados Celsius.
- `revisarTemperatura()`; Determina qué color debe mostrar el LED y a qué posición debe moverse el servomotor.
- `separarTemperatura()`; Divide la temperatura en decenas, unidades y decimal para los displays.
- `mostrarNumero()`; Activa los segmentos correspondientes al número deseado.
- `tareaDisplays()`; Realiza constantemente el multiplexado de los tres displays.
- `moverServo()`; Convierte los grados requeridos a una señal PWM.
- `escribirRGB()`; Controla la intensidad de los tres canales del LED RGB.
- `botonISR()`; Detecta mediante una interrupción cuándo se solicita una nueva medición.

Esta división permite que cada función tenga una responsabilidad específica y facilita realizar modificaciones o encontrar errores durante el desarrollo.

El proyecto permitió integrar lectura analógica, interrupciones, PWM, multiplexado de displays, multitarea mediante FreeRTOS y comunicación con una plataforma IoT. La transmisión de información mediante Adafruit IO permite complementar el sistema físico con monitoreo remoto y representación gráfica de las mediciones. En conjunto, el proyecto demuestra cómo diferentes herramientas de electrónica digital pueden combinarse para desarrollar un sistema de monitoreo similar a los utilizados en aplicaciones de cadena de frío.